

DOCKER

COMPLETE GUIDE TO DOCKER FOR BEGINNERS
AND INTERMEDIATES

Questions and Answers



DAY 7

MASTER DEVOPS WITH VINAY

61: How do you diagnose and troubleshoot a Docker container that has failed?

Answer:

1. Check Container Logs
 - Use `docker logs <container-id>` to view application output and identify errors or warnings.
2. Inspect the Container
 - Run `docker inspect <container-id>` to review configuration details, environment variables, and status.
3. Access the Container Shell
 - Use `docker exec -it <container-id> /bin/bash` (or `/bin/sh`) to enter the container and perform direct debugging.
4. Verify Resource Allocation
 - Check for adequate CPU, memory, and disk space to ensure the container isn't failing due to resource limits.

62: What is the process for debugging network connectivity issues in Docker containers?

Answer:

5. Inspect Docker Network Configuration
 - Use `docker network inspect <network-name>` to review network settings and connected containers.
6. Test Connectivity Between Containers
 - Run `docker exec -it <container-id> ping <target-container-name>` to check inter-container communication.
7. Check DNS Resolution
 - Use tools like `nslookup` or `dig` inside the container to verify name resolution.
8. Verify Port Mappings
 - Use `docker ps` to ensure the correct ports are exposed and mapped properly.
9. Inspect Firewall Rules
 - Confirm that host or container-level firewalls aren't blocking traffic between containers.

63: What steps can be taken to handle container failures or restart loops in Docker?

Answer:

10. Check Logs
 - Use `docker logs <container-id>` to identify error messages or application failures.
11. Inspect the Container
 - Run `docker inspect <container-id>` to review configuration, environment variables, and exit codes.
12. Use Restart Policies
 - Apply policies like `--restart=on-failure` when running containers to automatically recover from certain failures:
`docker run --restart=on-failure <image>`
4. Rebuild the Image
 - Review your Dockerfile, application code, and dependencies to ensure the image is built correctly and is stable at runtime.

64: What strategies do you use to resolve permission issues with Docker volumes?

Answer:

13. Adjust File Permissions
 - Use `chmod` or `chown` to set appropriate permissions on the host directory:
`chmod 777 /path/to/volume`
2. Run the Container as a Specific User
 - Avoid running as root by using the `--user` flag to match host file ownership:
`docker run --user 1000:1000 -v my-volume:/data my-app`
3. Validate Volume Mounts
 - Double-check that the correct host directory is being mounted and accessible to the container.

65: What is Docker's approach to creating multi-architecture container images?

Answer:

Docker uses Buildx to build and publish images that support multiple architectures (such as x86 and ARM), enabling portability across diverse systems.

Example Command:

```
docker buildx build --platform linux/amd64,linux/arm64 -t my-app:latest --push
```

This command builds the image for both amd64 and arm64 platforms and pushes it to the registry, making it usable across different hardware environments.

66: What Is Docker Swarm, and How Does It Differ from Kubernetes?

Answer:

Docker Swarm is Docker's native clustering and orchestration tool designed for simplicity and quick setup. In contrast, Kubernetes is a powerful, enterprise-grade orchestration platform with a broader feature set and ecosystem.

Key Differences:

Feature	Docker Swarm	Kubernetes
Complexity	Easy to set up and use	More complex setup and configuration
Scalability	Limited scalability	Highly scalable and production-ready
Ecosystem	Smaller ecosystem with fewer features	Rich ecosystem with extensive integrations and advanced capabilities

67: How do you use the docker logs command to debug container issues?

Answer:

The docker logs command is used to retrieve the output logs from a running or stopped container. It is essential for troubleshooting and debugging application errors or unexpected behavior within containers.

Basic Command:

```
docker logs <container-id>
```

Common Options:

1. View Logs in Real-Time:
`docker logs -f <container-id>`
2. Display the Last Few Lines:
`docker logs --tail 20 <container-id>`

68: What Is the Difference Between docker ps and docker inspect?

Answer:

Command and Purpose

docker ps: Lists running containers with basic details such as container ID, name, status, and exposed ports.

docker inspect: Provides detailed, low-level information about a container, image, or network in JSON format, including configuration, environment variables, and resource settings.

69: What is your approach to debugging build errors in a Dockerfile?

Answer:

1. Use Intermediate Layers
 - Insert RUN commands to inspect the state of files or environment variables during different build steps.
2. Build Specific Stages
 - Use the --target flag to build up to a particular stage in multi-stage Dockerfiles for easier debugging:
`docker build --target builder -t debug-image .`
3. Enable Verbose Output
 - Use --progress=plain to get detailed build logs and diagnose errors more effectively:
`docker build --progress=plain .`
4. Validate Dockerfile Syntax
 - Review your Dockerfile for syntax errors, typos, or incorrect commands before building.

70: How do you monitor and analyze the resource usage of a Docker container?

Answer: Use the docker stats command to monitor real-time resource usage of running containers.

Command: `docker stats`

Output Includes:

- CPU usage
- Memory consumption
- Network I/O
- Block I/O

To Monitor a Specific Container:

```
docker stats <container-id>
```

This helps identify performance bottlenecks and optimize container resource allocation.